Preprint · October 2026

Aggregation Hides Local Failure: Hierarchical, Evidence-Gated Burn-Rate Alerting for Global Edge Platforms

Pujitha Sri Lakshmi Paladugu
Independent Researcher, Lynnwood, Washington, USA
pujitha.rathna@gmail.com · ORCID 0009-0001-5678-4022

Summary

Global edge platforms serve traffic from dozens to hundreds of points of presence (PoPs), and most operators alert on service-level objectives (SLOs) computed over fleet-wide aggregates. We show that this practice is structurally blind to localized failures. For a PoP carrying a fraction w of traffic and failing with error rate e, the fleet-wide burn rate is approximately $w \cdot e/\beta$, where β is the error budget; a page fires only when this exceeds the burn threshold θ, so any PoP with $w < \theta\beta/e$ can fail indefinitely without paging. Under the widely used multi-window, multi-burn-rate rules ($\theta = 14.4$ for a 99.9% objective), a PoP returning errors on half of its requests must carry more than 2.9% of global traffic to be detected, and in a heavy-tailed fleet of 60 PoPs about 85% of PoPs fall below that bound. We propose hierarchical, evidence-gated burn-rate alerting, which keeps the fleet-wide rules and adds per-PoP rules guarded by a persistence condition, a Wilson lower-confidence bound on the PoP error ratio, and a minimum-failure count. In a reproducible simulation study (60 PoPs; 240 injected incidents; 168 simulated incident-free days), fleet-wide alerting detected 17% of single-PoP failures and none affecting the smallest quartile of PoPs, while the proposed scheme detected all of them with a median delay of 10 minutes and produced no false pages; naive per-PoP alerting detected them in 1 minute but generated 0.54 false pages per week. We give the analysis, the design, its mapping to common monitoring systems, and the simulation code.

Keywords: site reliability engineering; service-level objectives; error budgets; burn-rate alerting; edge computing; content delivery networks; observability; localized failure; gray failure

1 Introduction

Edge platforms terminate client connections, enforce security policy and route requests through geographically distributed points of presence (PoPs) connected by private backbones [1–3]. The distribution that gives these platforms their performance and resilience also makes them hard to

operate against reliability targets. A failure that affects one PoP, one region or one client population can be serious for the users it touches while remaining invisible in fleet-wide metrics; this is a specific instance of the broader class of *gray failures*, in which different observers of a system disagree about whether it is healthy [4].

The dominant practice for alerting on user-visible reliability is SLO-based burn-rate alerting, popularized by the Site Reliability Engineering literature [5,6]. An SLO sets a target for a service-level indicator such as the fraction of successful requests; the complement of the target is the error budget; and the burn rate is the speed at which the budget is being consumed. The multi-window, multi-burn-rate (MWMBR) scheme pages when both a long and a short window show the budget being consumed many times faster than sustainable, which balances detection speed against precision [6]. The scheme is elegant, and it is usually applied exactly as written: to the indicator of the service as a whole.

For a globally distributed edge, "the service as a whole" is an aggregate over PoPs whose traffic shares are highly unequal. This paper asks a simple question with uncomfortable consequences: *which PoP failures can fleet-wide burn-rate alerting detect at all?* We make four contributions.

1. **A dilution bound.** We derive a closed-form condition under which a PoP-local failure cannot trigger a fleet-wide burn-rate page, and the corresponding accrual delay for failures that can (Section 3). The bound shows that the blind spot is structural, not a tuning problem.
2. **A design.** We propose hierarchical, evidence-gated burn-rate alerting: fleet-wide MWMBR rules, plus per-PoP rules gated by persistence, a Wilson lower-confidence bound and a minimum-failure count, so that per-PoP alerting gains coverage without inheriting the noise of small samples and transient spikes (Section 4).
3. **A reproducible evaluation.** We evaluate four strategies in a simulation of a heavy-tailed, diurnally varying 60-PoP fleet with four classes of injected incidents and an incident-free control, reporting detection rate, time to detect, error budget consumed before detection, and false pages per week (Sections 5–6).
4. **Practical guidance.** We map the design onto common monitoring systems, give rules for choosing its parameters, and discuss its limits (Section 7). The simulation code accompanies the paper.

2 Background and related work

2.1 Service-level objectives and burn-rate alerting

A service-level indicator (SLI) is a quantitative measure of user-visible behaviour, such as the ratio of successful to valid requests; a service-level objective (SLO) is a target for that indicator over a compliance period, such as 99.9% over 30 days; and the error budget β is the permitted shortfall, here 0.1% of requests [5]. The burn rate over a window is the observed error ratio divided by β: a burn rate of 1 consumes exactly the budget over the compliance period, and a burn rate of 14.4 sustained for one hour consumes 2% of a 30-day budget. The SRE Workbook recommends paging when both a long window and a short window exceed a threshold, for example (1 hour and 5 minutes above 14.4) or (6 hours and 30 minutes above 6), and opening a ticket for slower burns [6]. The short window lets an alert reset quickly after recovery; the long window prevents brief blips from paging.

2.2 Edge platforms and localized failure

Content delivery and edge networks route clients to PoPs by DNS or anycast, and the population each PoP serves depends on network topology rather than on operator choice [1–3]. Measurement systems for such networks, such as Odin for Microsoft's CDN, exist precisely because client-perceived failures are often local to a network, a region or a PoP [3]. Data-centre monitoring systems such as Pingmesh made the same observation for latency inside the data centre: aggregate statistics hide the faults that matter [7]. Gray failure generalizes the point to cloud systems: the most damaging faults are frequently partial, visible to some observers and not others [4]. Tail-latency work makes a related argument about distributions: the mean conceals the tail that determines user experience [8].

2.3 Gap

The SRE literature describes burn-rate alerting for a service and notes that SLOs can be defined for different user journeys, but we are not aware of a quantitative treatment of how fleet-wide burn-rate rules behave when the service is a heavy-tailed aggregate of independently failing sites, nor of a principled way to add per-site alerting without multiplying false pages. Monitoring systems make per-site rules easy to write: a Prometheus rule evaluated per *pop* label, with a *for* clause that requires the condition to hold for a period before firing, is a few lines of configuration

[9]. What has been missing is the analysis that says when such rules are necessary, and evidence about how to gate them.

3 System model and the dilution bound

Consider a fleet of P PoPs. PoP i carries a fraction w_i of global valid requests ($\Sigma w_i = 1$) and, at a given time, fails a fraction e_i of them. With error budget β, the fleet-wide burn rate over a window is

$$b_g = \Sigma_i \, w_i \, e_i / \beta \qquad (1)$$

Suppose PoP j fails with error rate e while the rest of the fleet runs at a background rate $e_0 \ll e$. Then $b_g \approx w_j \, e/\beta + (1 - w_j) \, e_0/\beta$. A fleet-wide page with threshold θ requires $b_g > \theta$ in both windows of a rule. Ignoring the background term, the failure is undetectable by that rule, however long it lasts, whenever

$$w_j < \theta \, \beta / e \qquad (2)$$

For a 99.9% objective ($\beta = 0.001$) and the fast-burn threshold $\theta = 14.4$, a PoP that fails half of its requests ($e = 0.5$) is invisible to the one-hour rule unless it carries more than 2.88% of traffic, and a PoP that fails every request is invisible below 1.44%. The six-hour rule ($\theta = 6$) lowers the bound to 1.2% for $e = 0.5$, but only for failures that persist for hours. Heavy-tailed traffic makes the bound bite: in the log-normal fleets used in our study (Section 5), the median PoP carries about 1.1% of traffic and, averaged over 200 random fleets, 85% of PoPs carry less than 2.88%.

When a failure is detectable, the long window introduces an accrual delay. If the fleet-wide burn rate jumps from near zero to $b > \theta$ at time 0, the average over a long window of length L first exceeds θ at

$$t^* = L \cdot \theta / b \qquad (3)$$

A fleet-wide error rate of 2% ($b = 20$) therefore needs 43 minutes to trigger the one-hour rule. This is by design — the long window trades speed for precision — but it means that moderate fleet-wide incidents consume a measurable share of budget before paging, a property our simulation reproduces exactly (Section 6).

Equations (2) and (3) together explain the behaviour of fleet-wide burn-rate alerting on an edge: global incidents page, after a predictable delay; local incidents at small PoPs never page. The

second property is not fixed by lowering θ globally, because doing so raises the false-page rate for the whole fleet. It is fixed by evaluating the indicator where the failure is: per PoP.

4 Hierarchical, evidence-gated burn-rate alerting

Per-PoP alerting has an obvious cost. A small PoP serves few requests per window, so its error ratio is a noisy estimate; and a short burst of client misbehaviour at any PoP can drive its local burn rate far above threshold for a few minutes without any action being warranted. Naively applying MWMBR rules to every PoP therefore converts a blind spot into a stream of pages. We gate per-PoP rules with three conditions, each aimed at one source of noise.

- **Statistical evidence.** Let f and n be a PoP's failed and total requests in the short window. We require the Wilson score lower bound [10,11] on f/n at confidence z to exceed $\theta\beta$. Unlike the raw ratio, the bound shrinks toward zero when n is small, so a handful of failures at a tiny PoP cannot page on its own. We use $z = 3$ (one-sided ≈ 99.87%).
- **Minimum impact.** We require at least m failed requests in the long window ($m = 30$ in our study). This guards against pages whose absolute user impact is negligible.
- **Persistence.** We require the per-PoP condition to hold for K consecutive evaluation intervals ($K = 10$ minutes in our study). This suppresses transient spikes shorter than K, at the cost of delaying detection of real local failures by about K.

The resulting rule set is hierarchical: page if the fleet-wide MWMBR rule fires, *or* if any PoP satisfies its own MWMBR rule together with the three gates. The fleet-wide rule retains its role for global incidents; the per-PoP rule covers the region of the parameter space that Equation (2) shows to be invisible.

4.1 Mapping to monitoring systems

The design maps directly onto label-based monitoring systems. In Prometheus terms [9], recording rules compute per-PoP failed and total request counts over the short and long windows; the alerting rule compares the per-PoP burn rates and the Wilson bound against thresholds, with *count_over_time*-style minimum-failure conditions; and the rule's *for: 10m* clause implements persistence. The fleet-wide rule is unchanged. Two operational details matter. First, per-PoP rules multiply rule evaluation by P, which is negligible for tens or hundreds of PoPs but should be budgeted alongside existing per-PoP dashboards. Second, a per-PoP page should carry the PoP's

traffic share and the estimated budget impact, so that responders can distinguish a small, contained failure from the leading edge of a regional one.

4.2 Choosing parameters

The persistence window K is the main tuning knob and should be chosen from the operator's own telemetry: set K just above a high percentile (for example the 99th) of the duration of benign error spikes observed at individual PoPs, so that such spikes cannot page, and accept a detection delay of about K for real local failures. The Wilson confidence z and the minimum-failure count m matter most for fleets whose smallest PoPs serve only tens of requests per minute; for higher-volume fleets they rarely bind (Section 6.4).

5 Evaluation methodology

We evaluate the design by discrete-time simulation at one-minute resolution. The simulator, written in Python with NumPy, is deterministic given its random seeds and accompanies the paper; every number reported in Section 6 is produced by it. Table 1 summarizes the model.

Table 1. Simulation model and parameters.

Element	Setting
Fleet	60 PoPs; traffic shares drawn from a log-normal distribution ($\sigma = 1.0$) and normalized; median share ≈ 1.1%, smallest ≈ 0.1%, largest ≈ 11%
Load	200,000 requests/min fleet-wide on average; per-PoP diurnal sinusoid (±50%) with random phase (longitude proxy); per-minute counts ~ Poisson
Errors	Per-PoP background error rate uniform in [0.01%, 0.03%]; failed requests ~ Binomial
Benign spikes	Non-actionable error spikes at random PoPs, 6 per day on average, lasting 1–5 minutes at 5–20% error rate
Objective	99.9% availability over 30 days ($\beta = 0.001$)
Page rules	(1 h and 5 min burn > 14.4) or (6 h and 30 min burn > 6), as in [6]
Run length	5 days per incident trial (first 3 days warm the long windows); 17 days per incident-free run (14 measured)

We compare four strategies: **S1**, a static threshold paging when the fleet-wide five-minute error ratio exceeds 1%; **S2**, fleet-wide MWMBR; **S3**, MWMBR applied independently to every PoP; and **S4**, the proposed hierarchical, evidence-gated scheme with $K = 10$, $z = 3$ and $m = 30$. We inject four incident classes on day four of each trial, 60 trials per class (Table 2). An incident counts as detected if a page fires between its start and 30 minutes after its end; time to detect is measured from the start. We also report the error budget consumed by the affected PoPs before detection (or

over the whole incident, if undetected), as a percentage of those PoPs' own 30-day budget. False pages are measured on 12 incident-free runs (168 simulated days), counting distinct pages separated by at least 60 minutes.

Table 2. Injected incident classes (60 trials each; 60-minute duration).

Class	Description	Median affected traffic share
I1 Global	All PoPs fail 2% of requests (burn rate 20)	100%
I2 Regional	Five geographically adjacent PoPs fail 30% of requests	6.2%
I3 Single PoP	One randomly chosen PoP fails 50% of requests	1.0%
I4 Small PoP	One PoP from the smallest quartile fails 50% of requests	0.31%

Use of AI-assisted tools. In line with the journal's policy, the author discloses that an AI assistant (Claude, Anthropic) was used to help draft and edit the text and to write and debug the simulation code. All reported numbers are produced by the accompanying simulator. The author designed the study, reviewed and verified all content, code and results, and takes full responsibility for the article.

6 Results

6.1 Detection

Table 3 reports detection for each strategy and incident class. Fleet-wide strategies behave exactly as the analysis of Section 3 predicts. Both detect the global incident; S2 does so after a median 43 minutes, matching Equation (3) for $b = 20$. Neither detects a failure at a small PoP: S2 detected 17% of single-PoP failures (I3) and none of the small-PoP failures (I4). Undetected, a 60-minute small-PoP failure consumed a median 81% of that PoP's 30-day error budget. The regional incident (I2), which affects about 6% of traffic, sits near the bound: S2 detected only 53%, after a median 31 minutes, by which time a median 26% of the affected PoPs' budget had been consumed.

Table 3. Detection rate, median time to detect (TTD) and median error budget consumed by affected PoPs before detection, by strategy and incident class.

Class	S1 static	S2 global MWMBR	S3 per-PoP MWMBR	S4 proposed
I1 Global	100% · 2 min · 0.1%	100% · 43 min · 2.0%	100% · 39 min · 1.7%	100% · 42 min · 1.9%
I2 Regional	78% · 2 min · 1.5%	53% · 31 min · 26.0%	100% · 2 min · 1.2%	100% · 11 min · 6.3%
I3 Single PoP	27% · 2 min · 56.6%	17% · 33 min · 70.3%	100% · 1 min · 1.3%	100% · 10 min · 13.6%
I4 Small PoP	0% · — · 81.2%	0% · — · 81.2%	100% · 1 min · 1.4%	100% · 10 min · 14.0%

Budget is the median, over trials, of errors served by the affected PoPs before detection (or during the whole incident if undetected), as a percentage of those PoPs' 30-day error budget. TTD is the median over detected trials.

Per-PoP strategies detect every local incident. S3 does so fastest, within a minute. S4 detects every local incident after a median 10 minutes, the persistence window, and holds the budget consumed to about 14% of the affected PoP's monthly budget. The static threshold S1 is the fastest detector of the global incident but, like S2, misses most local ones.

6.2 The dilution curve

To isolate the effect of traffic share, we injected a 60-minute, 50%-error failure into a single PoP whose share was fixed at values from 0.2% to 8% (25 trials per value). Figure 1 shows detection probability. Fleet-wide MWMBR (S2) detects nothing below 1%, 8% of failures at 2%, 52% at 3% and 92% at 5%; the transition sits at the 2.88% predicted by Equation (2), smeared by diurnal variation in effective share. S4 detects every failure at every share.

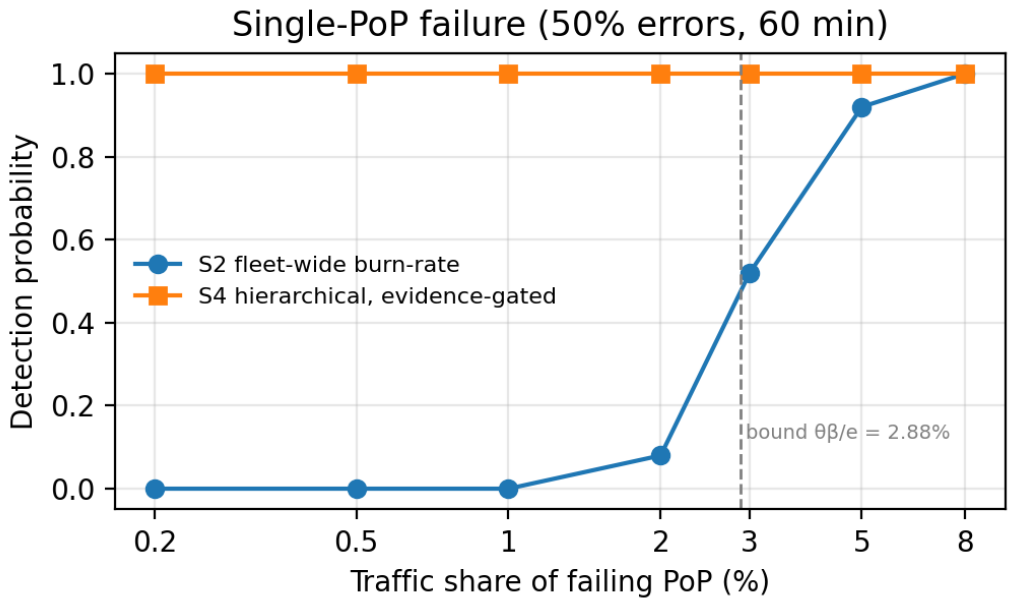


Figure 1. Probability of detecting a 60-minute, 50%-error failure at a single PoP, as a function of the PoP's traffic share (log scale). Fleet-wide burn-rate alerting (S2) undergoes the transition predicted by the dilution bound near 2.9%; the hierarchical scheme (S4) detects all cases.

6.3 False pages and the persistence trade-off

Over 168 incident-free simulated days, fleet-wide MWMBR produced no false pages, the static threshold produced 0.13 per week (consistent with benign spikes at the largest PoPs briefly lifting the fleet-wide ratio above 1%), and naive per-PoP MWMBR produced 0.54 per week (maximum 1.5 in a single two-week run). The proposed scheme produced none. Figure 2 shows where the improvement comes from. Sweeping the persistence window K for small-PoP failures, detection

stays at 100% for every K while median time to detect grows linearly with K; false pages remain at 0.44 per week for $K \le 5$ and fall to zero at $K = 10$. Because benign spikes in the model last at most five minutes, any K above that duration suppresses them; this is the basis of the tuning rule in Section 4.2.

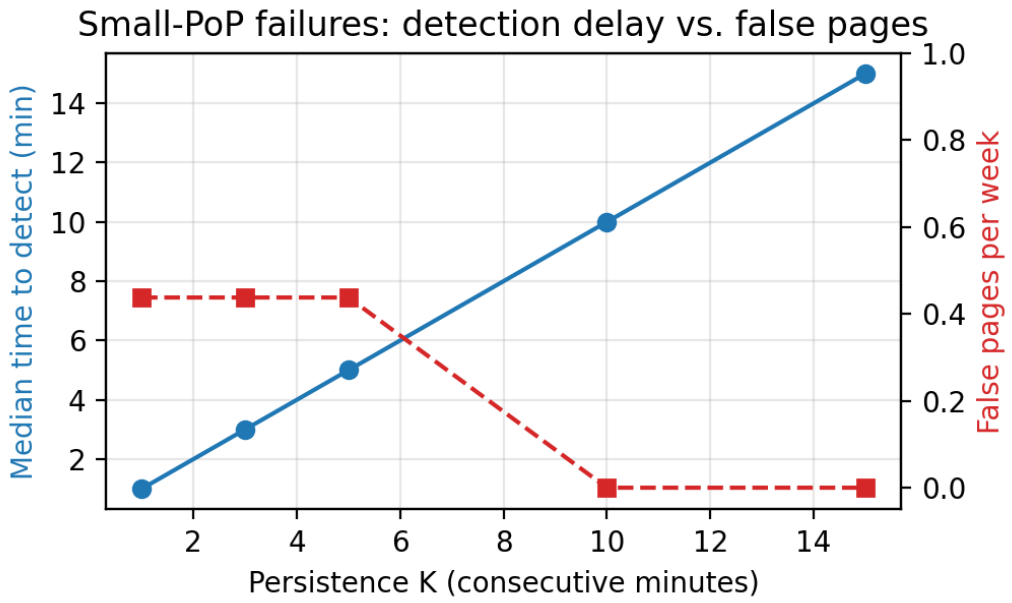


Figure 2. Effect of the persistence window K on small-PoP failures (I4): median time to detect (left axis) and false pages per week on incident-free runs (right axis).

6.4 Contribution of the statistical gates

At $K = 1$, S4 differs from S3 only by the Wilson bound and the minimum-failure count; these reduced false pages from 0.54 to 0.44 per week, a modest effect, because at the simulated volume most PoPs serve hundreds of requests per minute and benign spikes carry real statistical evidence. We repeated the false-page measurement on a fleet with one-tenth of the traffic (20,000 requests per minute fleet-wide). Naive per-PoP alerting produced 0.56 false pages per week and S4 none. In our model, persistence is the dominant control and the statistical gates are a safeguard that matters most at low volume; we report this plainly because it determines which parameters operators should spend effort tuning.

7 Discussion

7.1 Practical guidance

- **Check your fleet against the bound.** Compute $\theta\beta/e$ for your fast-burn threshold and the error rates you care about, and count how many PoPs fall below it. If many do, fleet-wide burn-rate alerting cannot protect their users.
- **Keep the fleet-wide rules.** They remain the right detector for global incidents, and they did not false-page in our study.
- **Add gated per-PoP rules rather than raw ones.** Choose the persistence window from the observed duration of benign spikes at individual PoPs; add the Wilson bound and a minimum-failure count if your smallest PoPs serve low volumes.
- **Page with context.** Include the PoP's traffic share and estimated budget impact in every per-PoP page, so responders can triage contained failures differently from emerging regional ones.
- **Re-validate with injected failures.** Inject synthetic failures at PoPs of different sizes and confirm that each is detected within the expected delay; the simulation in this paper is a template for that exercise.

7.2 Threats to validity

Our results come from simulation, not from production telemetry, and they inherit the model's assumptions. Errors are independent across requests given the PoP error rate; real failures can be bursty and correlated. Anycast or DNS steering can move users away from a failing PoP, changing traffic shares during an incident; we hold shares fixed, which makes local incidents slightly easier to detect than if traffic drained away. The benign-spike model determines the persistence window that suppresses false pages: the specific value $K = 10$ is a consequence of spikes lasting at most five minutes, and operators must measure their own distribution. We evaluate availability SLOs only; latency SLOs at the tail raise additional estimation issues that we do not address. Finally, minute resolution and the chosen windows follow common practice [6] but are not the only reasonable choices. Within these limits, the central finding does not depend on the simulation at all: Equation (2) is a property of averaging, and the simulation confirms it quantitatively.

7.3 Beyond availability

The same dilution argument applies to any SLI computed as a fleet-wide ratio, including TLS handshake success, web-application-firewall false-positive rates and, increasingly, the success rate of AI-inference requests routed through the edge. Wherever traffic is heavy-tailed across sites, the operator should expect the fleet-wide indicator to hide the failures of small sites, and should decide deliberately how to observe them.

8 Conclusion

Fleet-wide burn-rate alerting is the right tool for global incidents and the wrong tool for local ones. On a heavy-tailed edge, a PoP failing half of its requests is invisible to the standard fast-burn rule unless it carries almost 3% of traffic, and most PoPs do not. Hierarchical, evidence-gated alerting removes the blind spot at a cost of about one persistence window of detection delay, without the false pages of naive per-PoP rules. The analysis, design and simulation code are intended to let operators check their own fleets against the bound and adopt the scheme with parameters drawn from their own data.

Data availability statement

The simulation code (edge_slo_sim.py), the figure script and the raw results (results.json) that support the findings of this study are provided as supplementary material with this submission (edge_slo_sim_code.zip) and will be deposited in a public repository upon acceptance. No proprietary or production data were used.

Conflict of interest

The author declares no conflict of interest. This work was conducted independently, without funding, and does not use any employer data, systems or confidential information.

Use of AI-assisted tools

AI-assisted tool use is declared in Section 5 (Evaluation methodology), as required by the journal's policy.

References

[1] E. Nygren, R. K. Sitaraman, and J. Sun, "The Akamai network: a platform for high-performance internet applications," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 3, pp. 2–19, 2010.

[2] M. Calder, A. Flavel, E. Katz-Bassett, R. Mahajan, and J. Padhye, "Analyzing the performance of an anycast CDN," in *Proc. ACM Internet Measurement Conference (IMC)*, 2015, pp. 531–537.

[3] M. Calder, R. Gao, M. Schröder, R. Stewart, J. Padhye, R. Mahajan, G. Ananthanarayanan, and E. Katz-Bassett, "Odin: Microsoft's scalable fault-tolerant CDN measurement system," in *Proc. USENIX NSDI*, 2018, pp. 501–517.

[4] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao, "Gray failure: the Achilles' heel of cloud-scale systems," in *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, 2017, pp. 150–155.

[5] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.

[6] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, and S. Thorne, Eds., *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol, CA: O'Reilly Media, 2018, ch. 5, "Alerting on SLOs".

[7] C. Guo *et al.*, "Pingmesh: a large-scale system for data center network latency measurement and analysis," in *Proc. ACM SIGCOMM*, 2015, pp. 139–152.

[8] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[9] Prometheus Authors, "Alerting rules," Prometheus documentation, Cloud Native Computing Foundation. [Online]. Available: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

[10] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[11] A. Agresti and B. A. Coull, "Approximate is better than 'exact' for interval estimation of binomial proportions," *The American Statistician*, vol. 52, no. 2, pp. 119–126, 1998.

[12] B. H. Sigelman *et al.*, "Dapper, a large-scale distributed systems tracing infrastructure," Google, Tech. Rep., 2010.

[13] C. Adams *et al.*, "Monarch: Google's planet-scale in-memory time series database," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3181–3194, 2020.

[14] OpenTelemetry Authors, "OpenTelemetry specification," Cloud Native Computing Foundation. [Online]. Available: https://opentelemetry.io/docs/specs/otel/